

Práctica de Clase 2 (Hito 2 del TPO): Modelo de Dominio - Diagrama de Clases UML

Tema: Diseño del Modelo de Dominio, Diagramas de Clases UML, Relaciones entre Clases

Duración: 90 minutos **Herramienta:** Draw.io (diagrams.net), StarUML o similar

Objetivo

El objetivo de este Hito es traducir el análisis de dominio realizado en el Hito 1 (Casos de Uso, Actores, Atributos de Calidad) en un **Diagrama de Clases UML** que represente el Modelo de Dominio de LogiSmart. Este diagrama será la columna vertebral del diseño del sistema y guiará toda la implementación futura.

A diferencia del Hito 1 (que fue principalmente análisis), el Hito 2 requiere **decisiones de diseño** fundamentadas. No hay una única respuesta correcta. Lo que se evaluará es la calidad del modelo, la aplicación correcta de los conceptos de POO y, sobre todo, la **justificación** de cada decisión.

Contexto: LogiSmart Revisitado

Recuerden el caso de negocio de LogiSmart del Hito 1. Ahora, con el conocimiento de POO y Diagramas de Clases, deben diseñar la estructura interna del sistema. Las preguntas que deben responder son:

- ¿Cuáles son las **entidades principales** del dominio?
- ¿Qué **atributos** tiene cada entidad?
- ¿Qué **comportamiento** (métodos) debe tener cada entidad?
- ¿Cómo se **relacionan** estas entidades entre sí?
- ¿Cómo aplicamos **encapsulación** (visibilidad)?
- ¿Cómo aseguramos **alta cohesión** y **bajo acoplamiento**?

Actividades

Actividad 1: Identificación de Clases Candidatas

Objetivo: Extraer del análisis del Hito 1 las entidades principales que se convertirán en clases.

Proceso:

1. **Revisar los Casos de Uso del Hito 1:** Identifiquen todos los sustantivos importantes. Estos son candidatos a ser clases.
 - Ejemplo: "El Operador Logístico planifica rutas" → Candidatos: **Operador**, **Ruta**
 - Ejemplo: "El Cliente solicita un envío" → Candidatos: **Cliente**, **Envío**
2. **Filtrar Candidatos:** No todos los sustantivos son clases. Algunos son atributos de otras clases.
 - ¿Es "Dirección" una clase o un atributo de "Cliente"?
 - ¿Es "Ubicación" una clase o un atributo de "PuntoDeEntrega"?
 - Justifiquen sus decisiones.
3. **Documentar Candidatos:** Creen una lista de las clases principales con una breve descripción de cada una.

Preguntas Desafiantes:

- ¿Debería haber una clase **Usuario** base de la que hereden **Cliente** y **Operador**? ¿Por qué?
- ¿Es **Vehículo** una clase o un atributo de **Operador**? Justifiquen.
- ¿Debería haber una clase **Pago** separada o es un atributo de **Envío**?

Actividad 2: Definición de Atributos y Métodos

Objetivo: Para cada clase, definir su estado (atributos) y su comportamiento (métodos).

Proceso:

1. **Atributos:** Para cada clase, identifiquen qué datos necesita conocer.

- Ejemplo: Clase **Cliente** → atributos: **nombre**, **email**, **telefono**, **direccion**
- Apliquen **encapsulación**: todos los atributos deben ser **private**

2. **Métodos:** Identifiquen qué comportamiento debe tener cada clase.

- Ejemplo: Clase **Envío** → métodos: **cambiarEstado()**, **calcularCosto()**, **obtenerUbicacion()**
- Los métodos deben ser **public** (la interfaz del objeto)
- Pueden tener métodos **private** para lógica interna

3. **Visibilidad:** Apliquen correctamente los modificadores:

- **+** (**public**): Comportamiento accesible desde fuera
- **-** (**private**): Estado y lógica interna
- **#** (**protected**): Atributos/métodos compartidos con subclases (si aplica)

Preguntas:

- ¿Qué métodos debería tener la clase **Ruta**? ¿Debería calcular distancias? ¿Debería validar puntos de entrega?
- ¿Debería la clase **Envío** tener un método **calcularTiempoEstimado()**? ¿O eso es responsabilidad de otra clase?
- ¿Cuáles son los métodos **private** (lógica interna) vs **public** (interfaz)?

Actividad 3: Establecimiento de Relaciones

Objetivo: Dibujar las relaciones entre clases y justificar el tipo de relación.

Proceso:

1. **Identificar Relaciones:** Para cada par de clases, pregunten: ¿Se relacionan?

- ¿Un **Cliente** se relaciona con un **Envío**? ¿Cómo?
- ¿Un **Envío** se relaciona con una **Ruta**? ¿Cómo?
- ¿Un **Operador** se relaciona con un **Vehículo**? ¿Cómo?

2. **Determinar Tipo de Relación:**

- **Asociación:** ¿Simplemente se conocen?
- **Composición:** ¿Una es parte esencial de la otra?
- **Agregación:** ¿Una tiene a la otra pero pueden existir independientemente?
- **Herencia:** ¿Una es un tipo especializado de la otra?

3. **Multiplicidades:** Indiquen cuántos objetos de una clase se relacionan con cuántos de la otra.

- Un **Cliente** puede tener 0 a muchos (**0..***) **Envíos**
- Un **Envío** pertenece a exactamente 1 (**1**) **Cliente**

4. **Dibujar en Draw.io:** Creen el diagrama con todas las relaciones.

Preguntas:

- ¿Un **Envío** se **compone de** **PuntosDeEntrega** o se **asocia con** ellos?
- ¿Debería haber herencia entre **Usuario**, **Cliente** y **Operador**? ¿O es mejor composición?
- ¿Cuál es la multiplicidad correcta entre **Ruta** y **Envío**? ¿Una ruta tiene muchos envíos o un envío tiene muchas rutas?

Actividad 4: Análisis de Cohesión y Acoplamiento }

Objetivo: Revisar el diseño para asegurar que es mantenible y flexible.

Proceso:

1. **Cohesión:** Para cada clase, pregunten: ¿Tiene una única responsabilidad clara?
 - Si una clase tiene métodos para "gestionar envíos", "calcular impuestos" y "enviar emails", tiene baja cohesión.
 - Solución: Crear clases especializadas.
2. **Acoplamiento:** Pregunten: ¿Cuántas dependencias tiene cada clase?
 - Si cambio la clase **BaseDatos**, ¿cuántas otras clases se ven afectadas?
 - Si cambio la clase **Email**, ¿cuántas otras clases se ven afectadas?
 - Objetivo: Minimizar dependencias.
3. **Refactoring:** Si encuentran problemas, propongan mejoras.

Preguntas:

- ¿La clase **Envío** debería tener un método **guardarEnBaseDatos()**? ¿Por qué sí o por qué no?
- ¿La clase **Ruta** debería conocer detalles de cómo se calcula el costo? ¿O debería delegar eso a otra clase?
- ¿Hay clases que podrían fusionarse porque tienen responsabilidades similares?

Actividad 5: Documentación y Justificación

Objetivo: Documentar todas las decisiones de diseño.

Proceso:

1. Creen un documento que acompañe al diagrama
2. Para cada clase, expliquen:
 - ¿Por qué es una clase?
 - ¿Cuál es su responsabilidad principal?
 - ¿Por qué tiene esos atributos y métodos?
3. Para cada relación, expliquen:
 - ¿Por qué esa relación existe?
 - ¿Por qué es ese tipo de relación (asociación, composición, herencia)?
 - ¿Por qué esa multiplicidad?

Entregable

Un único archivo **HITO_2_DIAGRAMA_CLASES** que contenga:

1. Portada

- Nombre del Proyecto: LogiSmart
- Integrantes del grupo
- Fecha de entrega

2. Introducción

- Breve descripción del objetivo del Hito 2
- Resumen de las decisiones de diseño principales

3. Descripción de Clases

Una sección para cada clase con:

- **Nombre de la clase**
- **Responsabilidad principal:** ¿Cuál es su propósito en el sistema?
- **Atributos:** Listado con tipo y visibilidad
- **Métodos:** Listado con parámetros, tipo de retorno y visibilidad
- **Justificación:** ¿Por qué tiene esos atributos y métodos?

Ejemplo:

Clase: Envío

Responsabilidad Principal: Representar un envío individual que debe ser entregado.

Mantiene el estado del envío, su ubicación actual y su destino.

Atributos:

- `id: String` - Identificador único del envío
- `estado: EstadoEnvío` - Estado actual (pendiente, en tránsito, entregado)
- `origenUbicacion: Ubicacion` - Punto de origen
- `destinoUbicacion: Ubicacion` - Punto de destino
- `fechaCreacion: Date` - Cuándo se creó el envío
- `fechaEntrega: Date` - Cuándo se entregó (null si aún no)

Métodos:

- `+ cambiarEstado(nuevoEstado: EstadoEnvío): void` - Cambia el estado del envío
- `+ obtenerUbicacionActual(): Ubicacion` - Retorna la ubicación actual
- `+ calcularTiempoEstimado(): int` - Retorna minutos estimados hasta entrega
- `+ validarEstadoTransicion(nuevoEstado): boolean` - Valida que la transición sea válida

Justificación:

El envío es la entidad central del sistema. Necesita conocer su origen, destino y estado

actual para que el sistema pueda rastrearlo. Los métodos permiten cambiar su estado de forma controlada (solo transiciones válidas) y consultar información sobre él. El método ``validarEstadoTransicion()`` es privado porque es lógica interna de la clase.

4. Descripción de Relaciones

Una sección para cada relación importante con:

- **Clases involucradas**
- **Tipo de relación** (Asociación, Composición, Agregación, Herencia)
- **Multiplicidad**
- **Justificación**

Ejemplo:

Relación: Cliente - Envío

Tipo: Asociación

Multiplicidad: Un Cliente puede tener 0 a muchos Envíos (1 a 0..*)

Justificación: Un cliente puede solicitar múltiples envíos a lo largo del tiempo. Un envío pertenece a exactamente un cliente. Es una asociación simple (no composición) porque el envío puede existir sin el cliente (en el contexto del sistema, el envío es la entidad principal).

5. Diagrama de Clases UML

- Imagen del diagrama completo (exportado de Draw.io)
- Debe incluir todas las clases, atributos, métodos y relaciones
- Debe ser legible y bien organizado

6. Análisis de Cohesión y Acoplamiento

Una sección que analice:

- **Cohesión:** ¿Cada clase tiene una única responsabilidad clara?
- **Acoplamiento:** ¿Cuáles son las dependencias principales?
- **Problemas identificados:** ¿Hay clases con baja cohesión o alto acoplamiento?
- **Mejoras propuestas:** ¿Cómo podrían mejorarse?

Ejemplo:

Análisis de Cohesión

Clase Envío: Alta cohesión. Su única responsabilidad es representar y gestionar un envío individual. Todos sus métodos están relacionados con esa responsabilidad.

Clase Ruta: Media cohesión. Tiene responsabilidades de gestionar puntos de entrega y calcular distancias. Podría mejorarse creando una clase ``CalculadoraDeRutas`` separada.

Análisis de Acoplamiento

Dependencias principales:

- `Envío` depende de `Cliente` (necesita saber a quién pertenece)
- `Ruta` depende de `Envío` (necesita saber qué envíos contiene)
- `Operador` depende de `Vehículo` (necesita saber qué vehículos tiene)

Problemas: La clase `Ruta` depende directamente de `Envío`, lo que crea acoplamiento. Si cambia la estructura de `Envío`, tenemos que cambiar `Ruta`.

Mejora propuesta: Crear una interfaz `Entregable` que ambas implementen, reduciendo el acoplamiento.

7. Decisiones de Diseño Importantes

Una sección que documente decisiones no obvias:

- ¿Por qué eligieron herencia en lugar de composición para cierta relación?
- ¿Por qué ciertos métodos son privados?
- ¿Qué trade-offs tuvieron que hacer?

Recursos

- **Draw.io:** <https://www.diagrams.net/> (Gratuito, online)
- **StarUML:** <https://staruml.io/> (Versión gratuita disponible)
- **Notación UML:** <https://www.uml-diagrams.org/class-diagrams.html>
- **Ejemplos de Diagramas de Clases:** Buscar "UML Class Diagram Examples" en Google